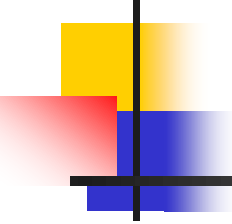


CS 2201: Turing Machines



Turing Machines are...

- Very powerful (abstract) machines that could simulate any modern day computer (although very, very slowly!)
- Why design such a machine?
 - If a problem cannot be “solved” even using a TM, then it implies that the problem is ***undecidable***
- Computability vs. Decidability

For every input,
answer YES or NO

A Turing Machine (TM)

- $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

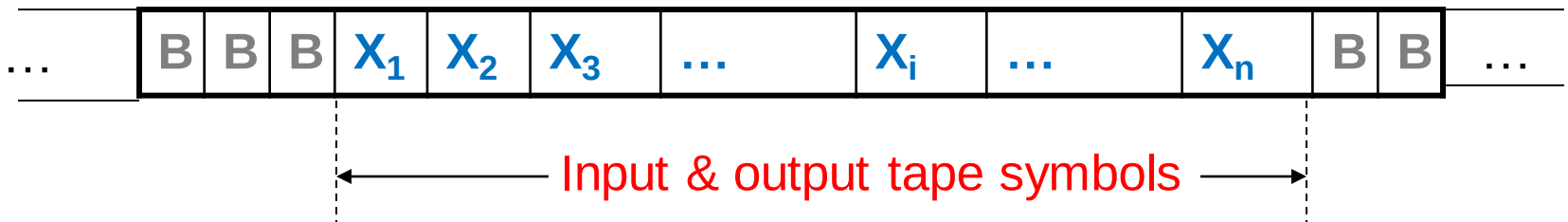
This is like the CPU & program counter

Finite control

Tape is the memory

Tape head

Infinite tape with tape symbols



B: blank symbol (special symbol reserved to indicate data boundary)

Transition function

You can also use:

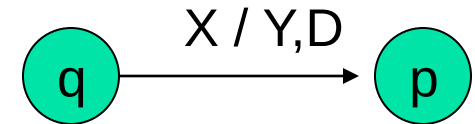
→ for R

← for L

- One move (denoted by |---) in a TM does the following:

- $\delta(q, X) = (p, Y, D)$

- q is the current state
- X is the current tape symbol pointed by tape head
- State changes from q to p
- After the move:
 - X is replaced with symbol Y
 - If D="L", the tape head moves "left" by one position.
Alternatively, if D="R" the tape head moves "right" by one position.



ID of a TM

- Instantaneous Description or ID :

- $X_1X_2\cdots X_{i-1}\textcolor{red}{q}X_iX_{i+1}\cdots X_n$

means:

- $\textcolor{red}{q}$ is the current state
 - Tape head is pointing to X_i
 - $X_1X_2\cdots X_{i-1}X_iX_{i+1}\cdots X_n$ are the current tape symbols

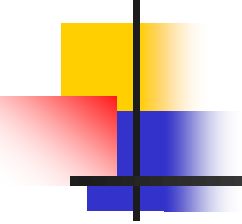
- $\delta(\textcolor{blue}{q}, X_i) = (\textcolor{red}{p}, \textcolor{violet}{Y}, R)$ is same as:

$$X_1\cdots X_{i-1}\textcolor{blue}{q}X_i\cdots X_n \quad | \text{---} \quad X_1\cdots X_{i-1}\textcolor{violet}{Y}\textcolor{red}{p}X_{i+1}\cdots X_n$$

- $\delta(\textcolor{blue}{q}, X_i) = (\textcolor{red}{p}, \textcolor{violet}{Y}, L)$ is same as:

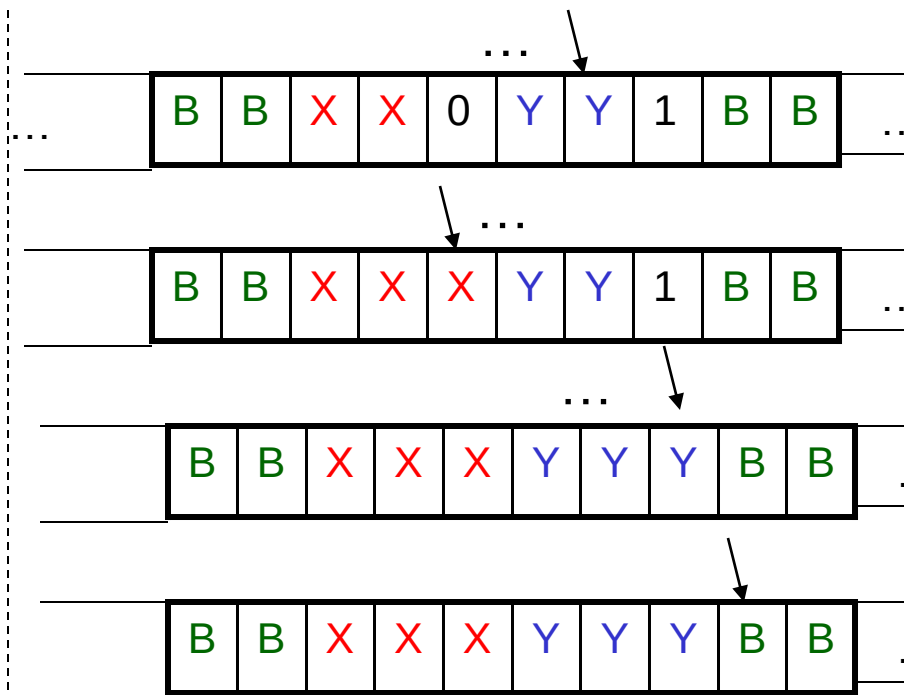
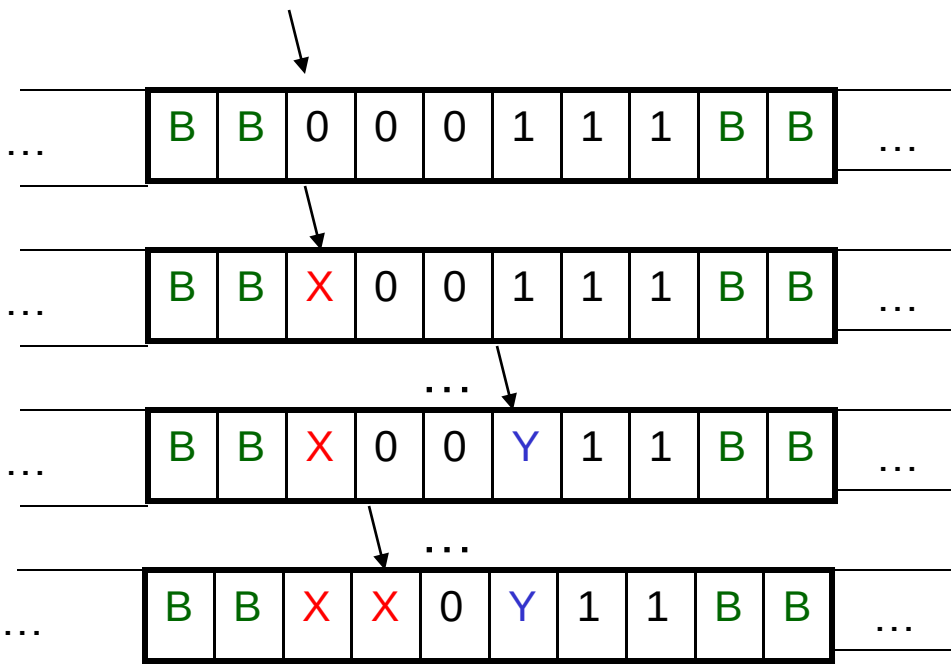
$$X_1\cdots X_{i-1}\textcolor{blue}{q}X_i\cdots X_n \quad | \text{---} \quad X_1\cdots \textcolor{red}{p}X_{i-1}\textcolor{violet}{Y}X_{i+1}\cdots X_n$$

Way to check for Membership

- 
-
- Is a string w accepted by a TM?
 - Initial condition:
 - The (whole) input string w is present in TM, preceded and followed by infinite blank symbols
 - Final acceptance:
 - Accept w if TM enters final state and halts
 - If TM halts and not final state, then reject

Example: $L = \{0^n 1^n \mid n \geq 1\}$

■ Strategy: $w = 000111$



Accept

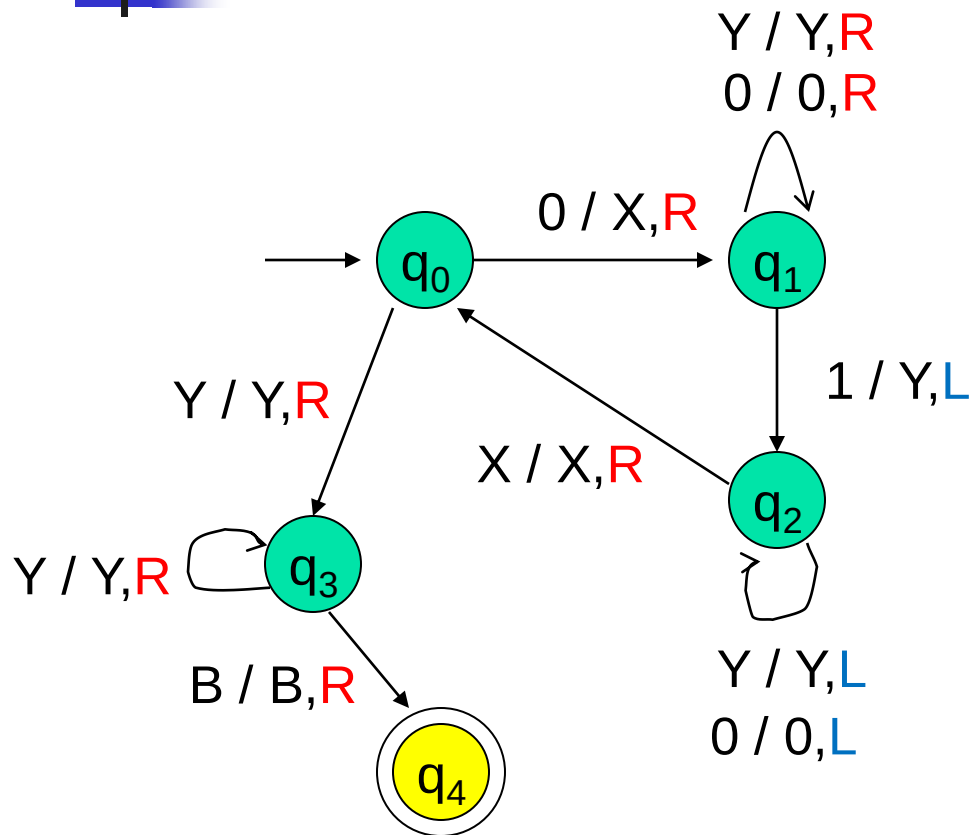
TM for $\{0^n 1^n \mid n \geq 1\}$

	Next Tape Symbol				
Curr. State	0	1	X	Y	B
→ q_0	(q_1, X, R)	-	-	(q_3, Y, R)	-
q_1	$(q_1, 0, R)$	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	$(q_2, 0, L)$	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	(q_4, B, R)
* q_4	-	--	-	-	-

Table representation of the state diagram

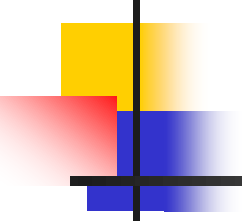
(q_3, Y, R) : When all the 0s are marked with X

TM for $\{0^n 1^n \mid n \geq 1\}$



1. Mark next unread 0 with X and move right
2. Move to the right all the way to the first unread 1, and mark it with Y
3. Move back (to the left) all the way to the last marked X, and then move one position to the right
4. If the next position is 0, then goto step 1.
Else move all the way to the right to ensure there are no excess 1s. If not move right to the next blank symbol and stop & accept.

Example 3: Multiplication



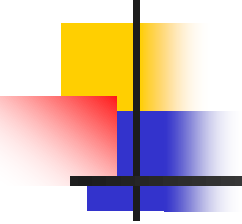
- $0^m 1 0^n 1$ (input), $0^{mn} 1$ (output)

- Pseudocode:

1. Move tape head back & forth such that for every 0 seen in 0^m , write n 0s to the right of the last delimiting 1
2. Once written, that zero is changed to B to get marked as finished
3. After completing on all m 0s, make the remaining n 0s and 1s also as Bs

Give state diagram

Calculations vs. Languages



A “calculation” is one that takes an input and outputs a value (or values)

The “language” for a certain calculation is the set of strings of the form “<input, output>”, where the output corresponds to a valid calculated value for the input

A “language” is a set of strings that meet certain criteria

E.g., The language L_{add} for the addition operation

“<0#0,0>”

“<0#1,1>”

...

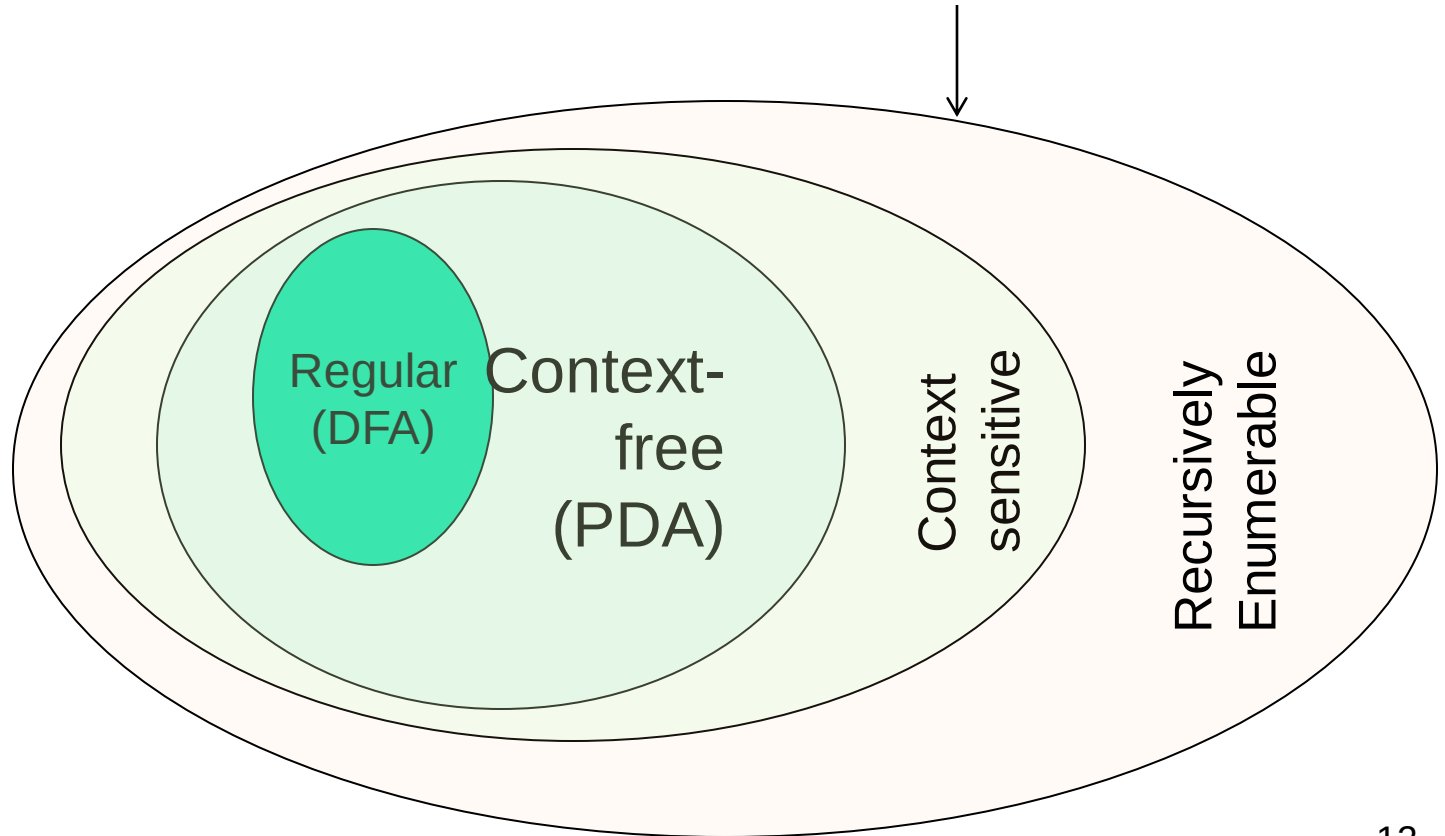
“<2#4,6>”

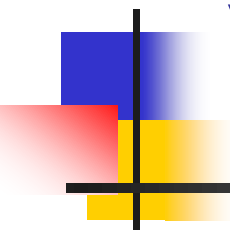
...

Membership question == verifying a solution
e.g., is “<15#12,27>” a member of L_{add} ?

Language of the Turing Machines

- *Recursive Enumerable (RE) language*





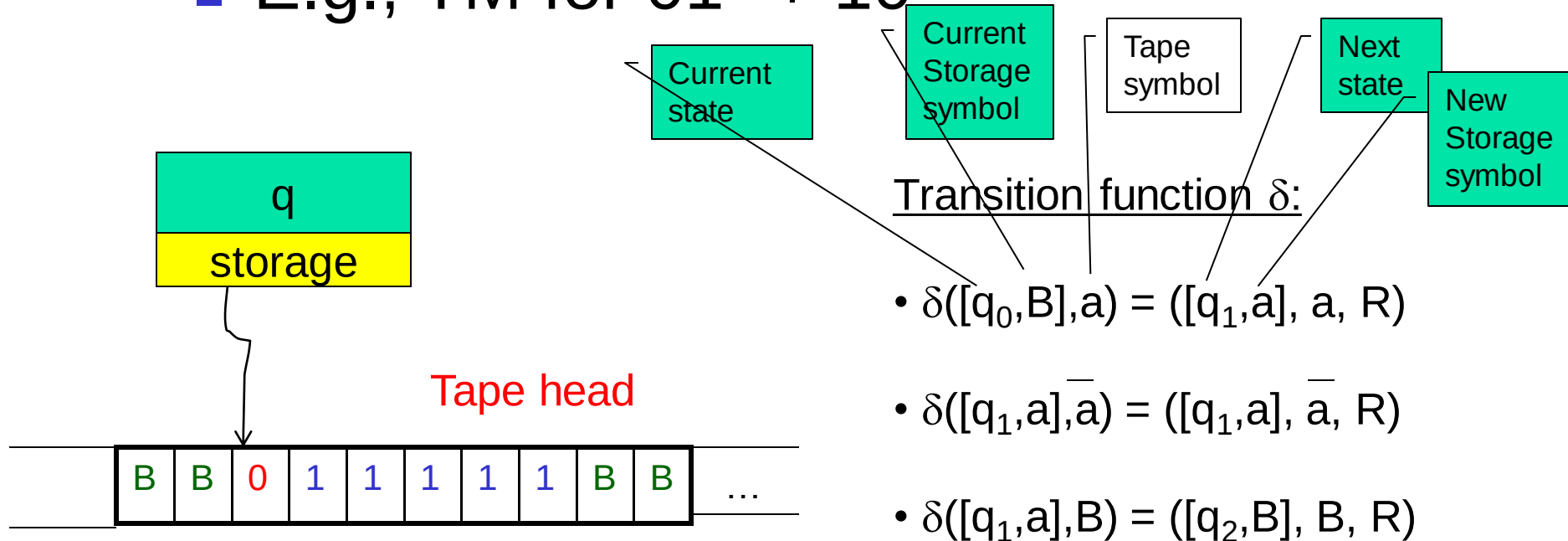
Variations of Turing Machines

TMs with *storage*

Generic description

Will work for both $a=0$ and $a=1$

- E.g., TM for $01^* + 10^*$



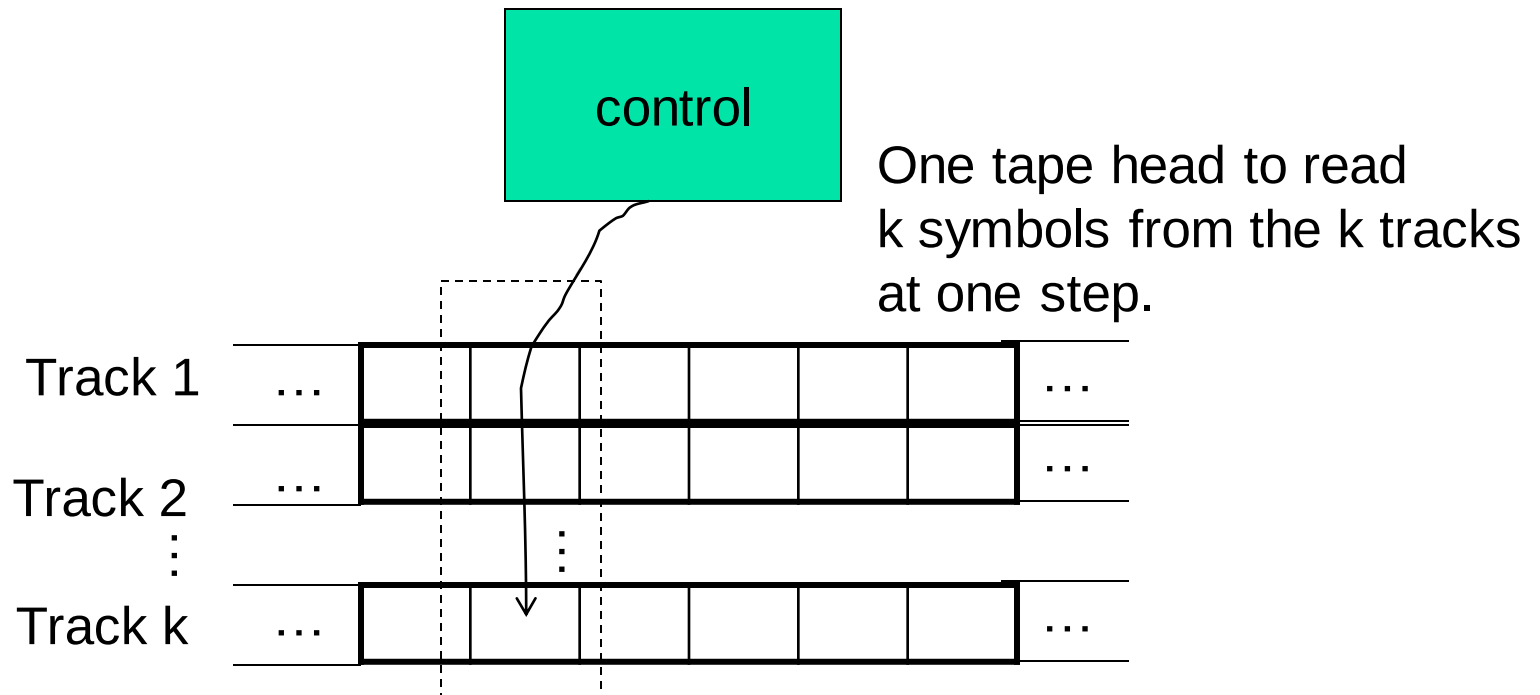
$[q, a]$: where q is current state,
 a is the symbol in storage

Are the standard TMs
equivalent to TMs with storage?

Yes

Multi-track Turing Machines

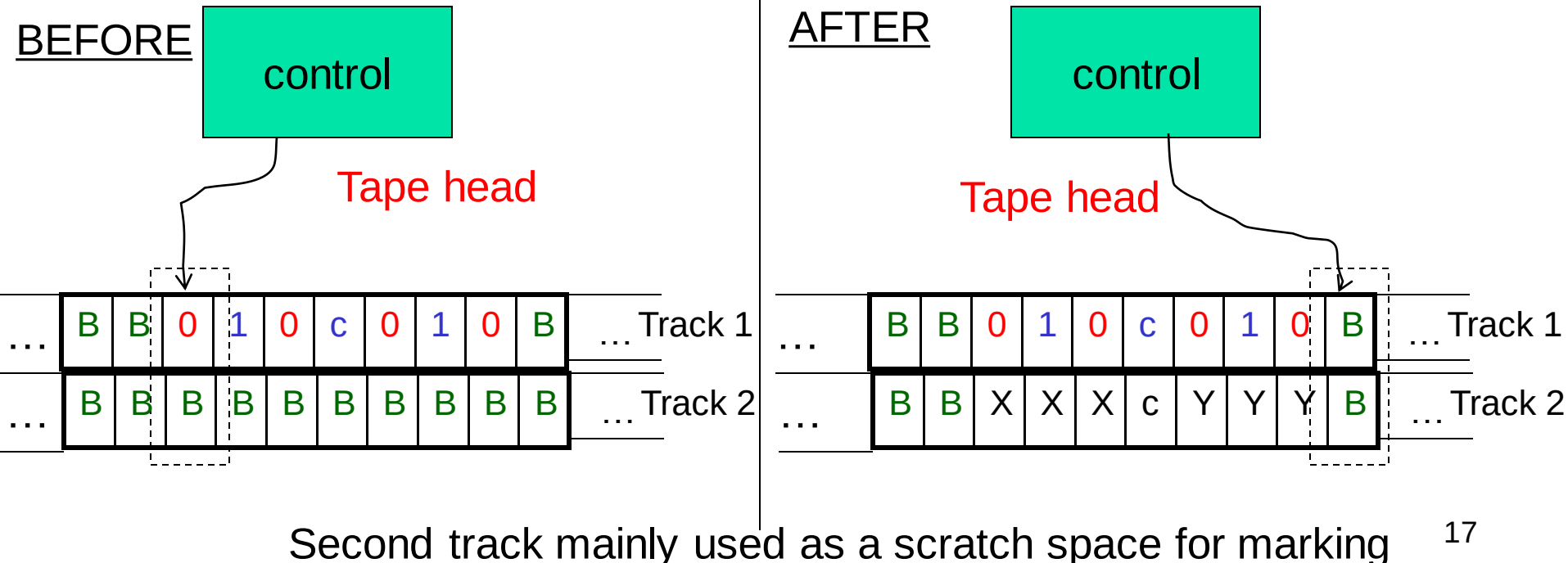
- TM with multiple tracks,
but just one unified tape head



Multi-Track TMs

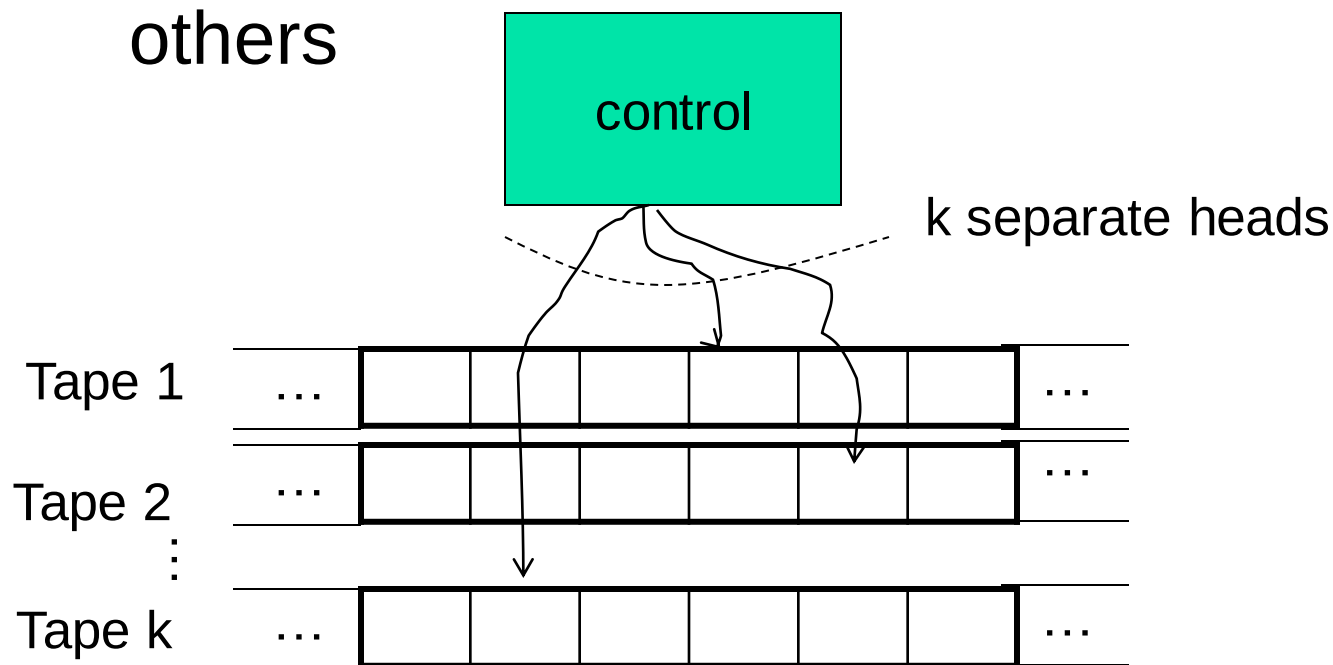
- TM with multiple “tracks” but just one head

E.g., TM for $\{wcw \mid w \in \{0,1\}^*\}$
but w/o modifying original input string



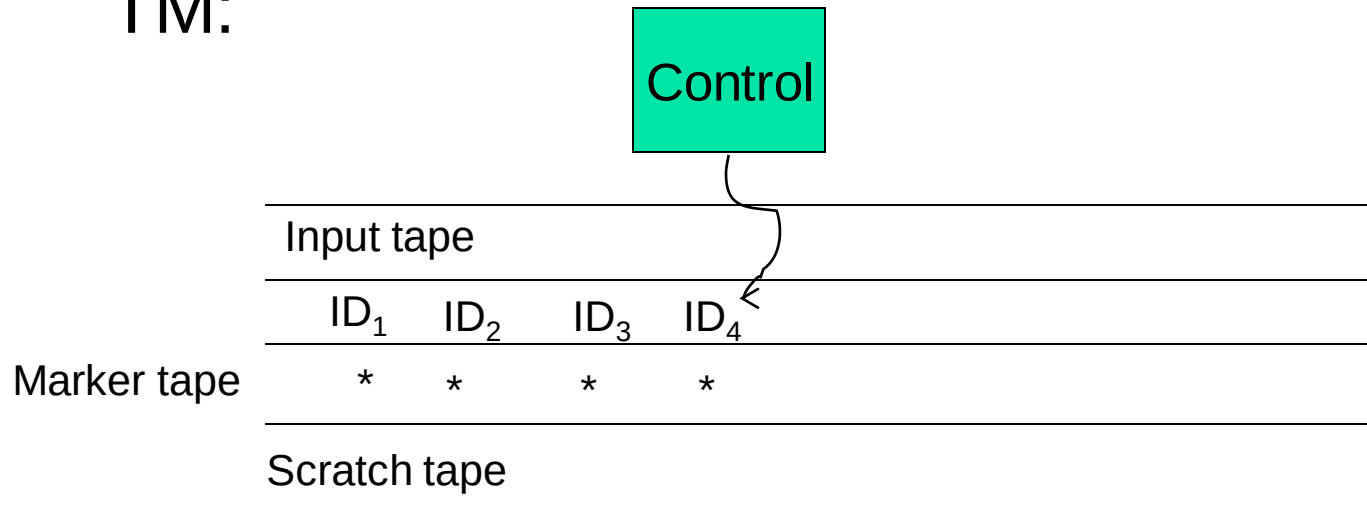
Multi-tape Turing Machines

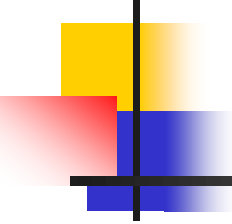
- TM with multiple tapes, *each tape with a separate head*
 - Each head can move independently of the others



Non-deterministic TMs

- A TM can have non-deterministic moves:
 - $\delta(q, X) = \{ (q_1, Y_1, D_1), (q_2, Y_2, D_2), \dots \}$
- Simulation using a multitape deterministic TM:





Summary

- TMs == Recursively Enumerable languages
- TMs can be used as both:
 - Language recognizers
 - Calculators/computers
- ***Basic TM is equivalent to all the below:***
 1. *TM + storage*
 2. *Multi-track TM*
 3. *Multi-tape TM*
 4. *Non-deterministic TM*
- TMs are like universal computing machines with unbounded storage